Unité 3: Programmation Python et premier script complet

Site: NiryoAcademy
Cours: De Blockly à Python avec le robot Ned2
Livre: Unité 3: Programmation Python et premier script complet

Imprimé par: Theo Bernicot
Date: vendredi 16 janvier 2026, 15:32

Table des matières

La différence entre le Python de NiryoStudio et PyNiryo

IDE pour Python

Utiliser Pyniryo

Les premiers codes avec Pyniryo

Créer ma première automatisation Python

Aller plus loin

La différence entre le Python de NiryoStudio et PyNiryo

Pourquoi y a-t-il deux façons différentes d'utiliser Python avec le robot ?

Pour comprendre cela, regardons d'abord **comment fonctionne le robot**.

À l'intérieur de la base du robot, il y a un **petit ordinateur (un Raspberry Pi)**.
Cet ordinateur est comme le “cerveau” du robot.
Ainsi, pour contrôler le robot, nous devons envoyer des ordres à ce cerveau.

Cas 1 : Lorsque vous utilisez **NiryoStudio**

- NiryoStudio est l'application officielle qui se connecte directement au cerveau du robot.
- Une fois connecté, vous pouvez utiliser **Blockly** ou le **Python ROS Wrapper** dans NiryoStudio.
- Dans ce cas, tout s'exécute **directement sur l'ordinateur du robot**.

Cas 2 : Lorsque vous utilisez un **IDE (PyCharm, VS Code, etc.)**

- La plupart des programmeurs préfèrent travailler dans un outil de codage spécial appelé **IDE**.
- Un IDE (Environnement de Développement Intégré) vous aide à écrire, tester et organiser votre code plus facilement.
- Mais ici, le code ne s'exécute pas directement *à l'intérieur* du robot. Il s'exécute sur votre propre ordinateur.

👉 Question : Comment votre code communique-t-il avec le cerveau du robot s'il ne s'exécute pas à l'intérieur ?

Réponse : Utiliser une **API**

Une **API (Interface de Programmation d'Applications)** est comme un **traducteur**.

- Imaginez que vous voulez demander à quelqu'un qui ne parle que japonais de jouer au football avec vous. Si vous ne parlez pas japonais, vous avez besoin d'un **traducteur** capable de répéter vos mots en japonais.
- Le robot fonctionne de la même manière : votre code Python ne “parle pas robot” directement. L'**API traduit** vos instructions dans le langage du robot.

Pour ce robot, l'API s'appelle **PyNiryo**.

Que fait exactement PyNiryo ?

- Avec PyNiryo, votre ordinateur se connecte au robot via le réseau.
- Vous pouvez ensuite envoyer des commandes Python depuis **n'importe quel IDE** (PyCharm, VS Code, etc.).
- L'API les traduit afin que le robot comprenne et exécute les mouvements.

✅ **Résumé** :

- **Méthode NiryoStudio** → Blockly / Python ROS Wrapper → s'exécute directement sur le cerveau du robot.
  *note : Vous pouvez aussi utiliser **Pyniryo** dans NiryoStudio. Cependant, lorsque vous convertissez un code Blockly en Python via la fonctionnalité de NiryoStudio, il sera converti en Python ROS Wrapper.*
- **Méthode IDE** → API PyNiryo → s'exécute sur votre ordinateur, communique via le réseau.

IDE pour Python

Pour le reste du TP, vous avez plusieurs options pour l'application que vous souhaitez utiliser lors du codage avec PyNiryo.

Option 1 : Utiliser **NiryoStudio**

Vous pouvez toujours utiliser NiryoStudio pour programmer avec PyNiryo.

- Vous n'avez pas besoin d'installer Python ou quoi que ce soit d'autre, car tout est déjà inclus dans NiryoStudio.
- Cependant, NiryoStudio n'est pas un outil complet de codage Python. Il est principalement conçu pour simplifier les choses.
- Cela signifie que vous pouvez écrire de petits programmes, mais vous remarquerez rapidement certaines limites. Par exemple, vous ne pouvez pas vraiment utiliser un terminal pour afficher des messages (`print`) ou pour demander des informations à l'utilisateur avec `input()`.

👉 NiryoStudio est parfait pour commencer, mais pas suffisant pour des projets plus importants.

Option 2 : Utiliser un **IDE externe** (recommandé)

C'est la méthode utilisée par la plupart des développeurs. Elle vous offre plus de liberté et plus d'outils pour travailler avec votre robot.

Étapes :

1. Installez Python sur votre ordinateur (la version 3.8 ou supérieure convient généralement).
2. **Choisissez un IDE** (Environnement de Développement Intégré). Deux choix populaires sont :
   - PyCharm
   - Visual Studio Code
3. **Installez la bibliothèque PyNiryo**.
   - Ouvrez un terminal (Invite de commandes sur Windows, ou Terminal sur Mac/Linux).
   - Tapez :
     ```
     pip install pyniryo
     ```
   - Cela téléchargera et installera les outils dont vous avez besoin pour communiquer avec le robot.

👉 Avec cette méthode, vous pouvez utiliser tout Python :

- Afficher des messages dans le terminal avec `print`.
- Demander des informations à l'utilisateur avec `input`.
- Organiser votre projet en plusieurs fichiers.
- Utiliser de nombreuses bibliothèques Python (math, données, IA, etc.).

C'est pourquoi l'option IDE externe est **recommandée** pour apprendre les pratiques de programmation réelles.

Utiliser Pyniryo

Créer un nouveau fichier Python

La première chose à faire lorsque vous voulez utiliser **PyNiryo** est d'indiquer à votre programme que vous allez l'utiliser.

En Python, lorsque vous voulez utiliser quelque chose qui ne fait pas partie du langage de base, vous devez **importer** cette chose. Importer, c'est comme dire :
"Hé Python, charge ces outils supplémentaires afin que je puisse les utiliser dans mon code."

Pour ce faire, en haut de votre fichier Python, écrivez :

```
import pyniryo
```

Maintenant, votre code sait que vous travaillez avec PyNiryo, et vous pouvez utiliser toutes les fonctions que cette bibliothèque fournit.

Comment savoir quelles fonctions existent ?

Dans Blockly, c'était très simple : vous aviez un panneau sur le côté avec tous les blocs que vous pouviez utiliser. Vous pouviez les faire glisser, déposer, et voir de petits modèles montrant comment les utiliser.

En Python avec une API comme PyNiryo, cela fonctionne un peu différemment. Vous n'avez pas de blocs colorés, mais vous consultez la **documentation**. La documentation est comme un grand menu de tout ce que la bibliothèque peut faire.

Vous devez toujours garder la documentation ouverte sur le côté pendant que vous codez. Ainsi vous pouvez vérifier :

- Quelles fonctions existent,
- Ce qu'elles font,
- Et comment les utiliser.

Voici la documentation PyNiryo : documentation PyNiryo

Dans cette documentation, vous trouverez :

- Comment installer la bibliothèque sur Python,
- Quelques exemples simples pour débuter,
- Et la liste complète des fonctions avec explications.

Exemple : Connexion au robot

Comme nous l'avons dit plus tôt, pour contrôler le robot, vous devez d'abord vous connecter à celui-ci. Voici le code qui fait cela avec PyNiryo :

```
import pyniryo

robot = pyniryo.NiryoRobot("10.10.10.10")  # remplacez l'adresse IP par celle de votre robot
```

**Vous pouvez noter l'utilisation du "#" pour écrire un commentaire dans mon code Python*

Ce code fait deux choses :

1. Il se connecte au robot.
2. Il stocke cette connexion dans une variable appelée `robot`.

Plus tard, lorsque vous voudrez faire bouger le robot, vous utiliserez cette variable `robot` pour envoyer des commandes.

Remarquez quelque chose : nous avons dû écrire `pyniryo.NiryoRobot`. C'est parce que **NiryoRobot** ne fait pas partie du langage Python de base. Il provient de la bibliothèque PyNiryo. En écrivant `pyniryo.` devant, Python sait qu'il doit chercher dans les outils PyNiryo.

Une façon plus courte d'importer

Il existe une autre manière de faire cela, qui rend le code un peu plus propre. Au lieu d'importer toute la bibliothèque PyNiryo, vous pouvez importer uniquement la partie dont vous avez besoin.

Au début de votre fichier, écrivez :

```
from pyniryo import NiryoRobot

robot = NiryoRobot("192.168.1.192")  # remplacez par l'adresse IP de votre robot
```

Maintenant, vous n'avez plus besoin d'écrire `pyniryo.` à chaque fois. Vous pouvez simplement écrire `NiryoRobot` directement, car vous avez indiqué à Python exactement quel outil de la bibliothèque vous vouliez charger.

Résumé en mots simples :

- `import` = dire à Python quels outils supplémentaires vous voulez utiliser.
- Documentation = votre “menu” de toutes les fonctions que vous pouvez appeler.
- `pyniryo.NiryoRobot` = la façon longue.
- `from pyniryo import NiryoRobot` = la façon courte.

Les premiers codes avec Pyniryo

Une bonne façon d'apprendre à utiliser une bibliothèque Python spécifique est de regarder les exemples existants, de les essayer, et à partir de là, d'imaginer les possibilités.

Donc, suivez les exemples fournis dans la documentation jusqu'à la section Exemple : Vision.

Pour chaque exemple, adaptez le code pour produire un résultat concret.

Exemples : Bases

Exemples : Action de l'outil

Exemples : Convoyeur

Exemples : Vision

Créer ma première automatisation Python

Maintenant que vous avez toutes les connaissances nécessaires, il est temps de créer votre première **automatisation Python** avec le **robot Ned2** !

Cette automatisation consistera en **deux processus principaux** :

1. **Remplissage du convoyeur** avec des objets (cercles et carrés) provenant du lot.
2. **Vidage du convoyeur** en utilisant le système de vision.

Processus 1 : Remplissage du convoyeur

À cette étape, vous allez charger le convoyeur de la simulation avec les objets disponibles.

1. Créez une fonction `place_on_conveyor()`

- **Paramètres :** la position (pose) de l'objet à saisir.
- **Retour :** rien.
- **Description :**
  La fonction doit :
  - Saisir l'objet depuis la pose spécifiée.
  - Le placer au **début du convoyeur**.
  - Faire avancer légèrement le convoyeur pour laisser de la place au prochain objet.

2. Créez une fonction `fill_up_conveyor()`

- **Paramètres :** une liste d'objets à placer sur le convoyeur.
- **Retour :** rien.
- **Description :**
  La fonction doit :
  - Appeler `place_on_conveyor()` pour chaque objet de la liste,
  - S'assurant que le convoyeur est rempli avec tous les objets du lot.

Processus 2 : Vidage du convoyeur avec Vision

Maintenant, utilisons le **système de vision** du robot pour détecter et saisir automatiquement les objets sur le convoyeur.

Créez une fonction `empty_conveyor()`

- **Paramètres :** le nombre d'objets à retirer (vous pouvez utiliser la fonction Python `len()` pour obtenir la taille d'une liste).
- **Retour :** rien.
- **Description :**
  Pour chaque objet, le robot doit :
  1. pose d'observation.

2. Faire avancer le **convoyeur** jusqu'à ce qu'un objet soit détecté dans l'espace de vision, puis l'arrêter.
3. Effectuer un **saisie par vision**.
4. Placer l'objet sur la **plateforme de vision** (utilisée comme zone de stockage).

Télécharger un exemple de solution:

Code Solution

Aller plus loin

Félicitations ! 🎉
Vous venez de faire vos premiers pas du **programmation visuelle** avec Blockly vers le **codage textuel** en Python, et c'est un grand pas en avant.

Avec Python, vous avez maintenant un **contrôle total** sur le comportement du robot :

- Vous pouvez écrire des mouvements précis, des boucles et des conditions.
- Vous pouvez intégrer des capteurs, des caméras et le traitement de données.
- Vous pouvez connecter le robot à des systèmes externes, comme des bases de données, des serveurs, ou même l'Internet des Objets (IoT).

Mais ce n'est que le début. Python ouvre la porte à la **robotique et à l'intelligence artificielle réelles** :
Vision par ordinateur – Reconnaître et suivre des objets en utilisant des bibliothèques comme `OpenCV` ou TensorFlow.
Apprentissage automatique – Apprendre au robot à tirer des leçons de l'expérience et à prendre des décisions.
Réseaux & IoT – Contrôler les robots à distance ou les faire communiquer avec d'autres appareils.
Interfaces & Jeux – Créer des panneaux de contrôle interactifs avec `pygame` ou des tableaux de bord web avec `Flask`.
Automatisation & Industrie 4.0 – Combiner la robotique avec l'analyse de données et le cloud computing.

Chaque commande Python que vous avez apprise vous donne un **pouvoir créatif supplémentaire** pour concevoir, expérimenter et innover.